

Entwicklung eines klassischen Kreuzungsdetektors

Studienarbeit

Studiengang Informatik

Duale Hochschule Baden-Württemberg Stuttgart

Reinhold Brant

Eingereicht am: 22.03.2026

Matrikelnummer, Kurs: 7261389, STG-TINF23ITA

Unternehmen: Robert Bosch GmbH, 76131, Karlsruhe

Betreuer im Unternehmen: John Appleseed

Betreuer an der DHBW: Prof. Dr. Daniel Düsentrieb

Inhalt

1 Packages to use	1
2 Abstract	2
3 Einleitung	3
3.1 Problemstellung	3
3.2 Kreuzungen	4
4 Technischer Kontext	5
5 Modulübersicht	6
6 Architektur und Datenfluss	7
7 Entwicklungs- und Evaluationsprozess	8
7.1 Bildvorverarbeitung	8
7.2 Kantenerkennung	8
7.2.1 Canny Edge Detector	8
7.3 Linienextraktion	10
7.3.1 Line Segment Detector	10
7.3.2 Hough Transformation	11
7.3.3 Pipeline für die Linienerkennung	14
7.4 Linienvorverarbeitung	14
7.4.1 Linienlängenfilter	14
7.4.2 Region of Interest Filter	15
7.4.3 Linienfusion	16
7.4.4 Winkelfilter	18
7.4.5 Pipeline für die Linienvorverarbeitung	18
7.5 Haltelinienklassifikation	19
7.5.1 Primitive Ermittlung der Ego- und Opplinie	19
7.5.2 Kreuzungsmittenbasierte Detektion	20
7.5.3 Sektorbasierte Detektion der rechten und linken Haltelinie	20
7.5.4 Zentrumsbasierte Detektion aller Haltelinien	22
7.6 Berechnung des Kreuzungszentrums	22
7.6.1 Primitive Bestimmung der Kreuzungsmitte	22
7.6.2 Algorithmische Kreuzungsmittendetektion	23
7.6.3 Hybrider Ansatz zur Kreuzungsmittenberechnung	26

7.6.4 Pipeline zur Kreuzungszentrumserkennung	26
7.6.5 Erweiterung der Kreuzungsmitte	27
7.7 Klassifizierung der Linienart	27
7.7.1 Linienausschnitt	28
7.7.2 Weißwertberechnung	28
7.7.3 Gap-Detektion	30
7.7.4 Pipeline zur Klassifizierung der Linienart	31
7.8 Heading der Straße	32
7.9 Plausibilisierung der Ego- und Opplinie	32
7.10 Plausibilisierung der Querlinien	32
7.11 Ergebnisaggregator	32
8 Erläuterungen	33
8.1 Ausdrücke und Abkürzungen	33
8.2 Listen	33
8.3 Abbildungen und Tabellen	34
8.3.1 Abbildungen	34
8.3.2 Tabellen	34
8.4 Programm Quellcode	34
8.5 Verweise	35
9 Fazit	36
A Literatur	37
B Glossar	38



Packages to use

- fletcher
- lovelace
- pavemat (für image darstellungen und ähnliches mit pixeln)
- showybox für theoreme und so
- lilaq (data visu)
- zebraw for code



Abstract

Bei der CAuDri-Challenge - CAuDri steht für Cognitive Autonomous Driving - treffen sich verschiedene Hochschulgruppen aus ganz Deutschland, um ihre autonomen Modellfahrzeuge in zwei Disziplinen, dem Free Drive und dem Obstacle Evasion, gegeneinander antreten zu lassen. Dabei muss das Fahrzeug auch Kreuzungen durchfahren können. In der aktuellen Version des Fahrzeugs werden Schilder an der Kreuzung erkannt und nach einer bestimmten Strecke hinter diesen gestoppt, um kurz vor der Haltelinie zum Stehen zu kommen. Eine Verbesserung dieses Systems ist eine Haltelinienerkennung. Diese ermöglicht ein präzisen Haltevorgang an der Haltelinie. Zusätzlich erfordern es die aktuellen Disziplinen der CAuDri-Challenge nur, an der Kreuzung den Spuren der Vorfahrtsstraße zu folgen. In einer neuen Disziplin der CAuDri-Challenge sollen aber beliebige Abbiegevorgänge an der Kreuzung möglich sein.

Ziel der Arbeit ist, eine Haltelinien- und Kreuzungserkennung zu entwickeln, die Abbiegespuren zuverlässig in vielen Situation erkennt. Außerdem soll es die Haltelinienerkennung ermöglichen, exakt an der Haltelinie zum Stehen zu kommen. Das gesamte Modul soll eine regelkonforme Durchfahrt in alle Kreuzungsrichtungen zuverlässig ermöglichen. Das Modul muss unabhängig vom Untergrund beziehungsweise anderen Störquellen robust und zuverlässig Haltelinien und Spuren erkennen.



Einleitung

3.1 Problemstellung

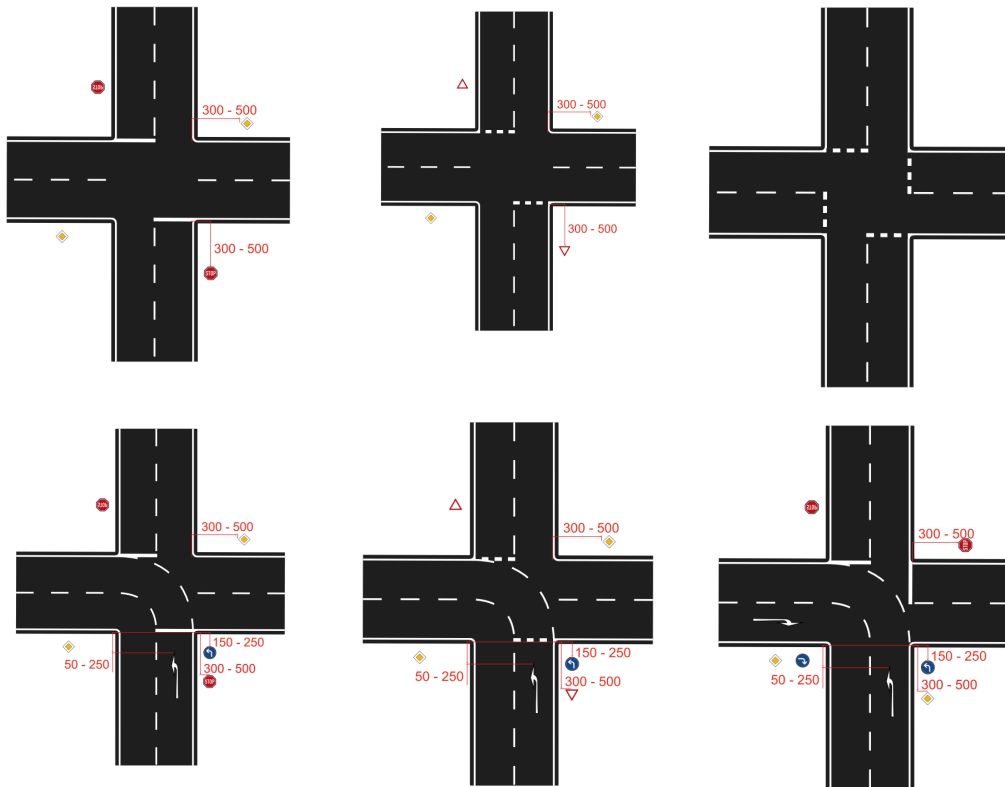
In der Caudri Challenge durchfährt das Fahrzeug autonom eine Strecke. Dabei nutzt es eine Kamera, um zu navigieren. In der Disziplin Free Drive muss das Fahrzeug so lange wie möglich ohne Steuerung von außen auch Kreuzungen korrekt erkennen und durchfahren. Um dies zu erreichen, muss die Kreuzungserkennung alle Haltelinien innerhalb der Kreuzung korrekt erkennen und anhand dieser die Kreuzung klassifizieren, bevor das Fahrzeug diese durchfahren kann.

Das Modul gibt die gewonnenen Informationen über die Kreuzung nach der Berechnung an das Pfadplanungsmodul weiter, welches anschließend den korrekten Fahrtweg durch die Kreuzung berechnet.

Das Modul wird als klassische Kreuzungserkennung implementiert und nutzt dabei keine Deep Learning Verfahren. Das hat einige Vorteile für das Deployment des Moduls auf dem Fahrzeug. Zum Einen sind Ergebnisse des Moduls einfach zu interpretieren, da die Berechnungen des Moduls vollständig regelbasiert getroffen und nicht durch ein Modell berechnet werden. Außerdem können Parameter wie etwa für Winkelfilter, Längendifter und andere regelbasierte Systeme besser angepasst werden, um Ergebnisse zu optimieren. Dazu wäre für einen Deep Learning Ansatz ein aufbereiteter Datensatz mit sauberen Labels notwendig, was durch den klassischen Ansatz wegfällt.

3.2 Kreuzungen

Um eine zuverlässige Kreuzungserkennung auf dem Fahrzeug zu implementieren werden zuerst die Kreuzungen betrachtet, die das System erkennen soll. Innerhalb der Caudri-Challenge gibt es eine Menge an Kreuzungen, die sich durch die Position der Haltelinien, der Art der Haltelinien und der Straßenführung innerhalb der Kreuzung unterscheiden. Diese sind dokumentiert.



Die zu erkennenden Kreuzungen sind folgendende:

- Ego- und Gegenspur Haltelinien durchgezogen
- Ego- und Gegenspur Haltelinie gestrichelt
- Gestrichelte Haltelinien an allen Spuren der Kreuzung
- Abknickende Vorfahrtsstraße links mit durchgezogenen Linien in der Ego- und Gegenspur
- Abknickende Vorfahrtsstraße links mit gestrichelten Haltelinien in der Ego- und Gegenspur
- Abknickende Vorfahrtsstraße links mit durchgezogener Haltelinie in der Gegenspur und der rechten Spur

Die zu erkennenden Kreuzungen sind standardisiert und sehen innerhalb der Strecke stets gleich aus.



Technischer Kontext



Modulübersicht

6

Architektur und Datenfluss



Entwicklungs- und Evaluationsprozess

Um aus einem Rohbild aus der `camera_preprocessing` Node Informationen zu gewinnen, wird das Rohbild innerhalb einer Pipeline verarbeitet. Innerhalb der Pipeline sind mehrere Designentscheidungen zu treffen, um ein bestmögliches Ergebnis zu erzielen. Der Entwicklungsprozess wird in Iterationen durchgeführt, nach der jeder das Ergebnis der Pipeline evaluiert und getestet wird.

7.1 Bildvorverarbeitung

TODO mit blurring und dem ganzen spaß

7.2 Kantenerkennung

Ziel des ersten Entwicklungsschritts ist das Extrahieren von Linien aus dem Rohbild. Dafür werden verschiedene Methodiken betrachtet, mit denen aus dem Rohbild eine Menge an Linien extrahiert werden kann, die dann in weiteren Schritten zum Bestimmen der Ego- und Opp-Haltelinien genutzt werden können.

7.2.1 Canny Edge Detector

Bevor Linien aus dem Rohbild extrahiert werden können, muss dieses erstmal vorverarbeitet werden. Der erste Verarbeitungsschritt ist es, aus dem Rohbild Kanten zu detektieren. Kantendetektion ist ein Verfahren, bei dem versucht wird, die Kanten

von Flächen innerhalb eines Bildes zu detektieren, indem Stellen innerhalb des Bildes ermittelt werden, an denen die Intensitätsänderung der Pixelhelligkeiten besonders stark ist [1] [S.53].

Der Canny Edge Detection Algorithmus besteht aus mehreren Schritten. Das Rohbild ist dabei ein Graustufenbild und nicht farbig.

1. Im ersten Schritt wird das Bild entrauscht. Dafür wird der Gaussian Blur (dt. Gaußscher Weichzeichner) auf das gesamte Bild angewandt. Dieser wird nach folgender Vorschrift durchgeführt $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$.
2. Nun das Bild mit zwei Matrizen G_x und G_y , den so genannten Sobel-Filtern gefaltet. Diese sind wie folgt definiert.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (1) \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad (2)$$

Die Faltung mit diesen zwei Matrizen resultiert in einem neuen Bild, welches an den Stellen, an denen der Gradient der Pixelwerte zwischen benachbarten Pixeln am größten ist, hellere Pixel hat als an Stellen, wo der Gradient nicht so stark ist. Benachbarte Pixel, die alle den selben Wert haben, werden nach der Faltung einen Pixelwert von 0 haben, weswegen sie schwarz sind. Da sowohl entlang der x-Achse als auch der y-Achse gefaltet wird, wird jedem Pixel somit ein Vektor der Größe 2x1, welcher die Intensität der Änderung in der respektiven Richtung anzeigt.

3. Die Werte aus dem Spaltenvektor mit den Intensitätswerten des Gradienten werden nun benutzt, um die Richtung der Kante zu bestimmen. Die Richtung der Änderung wird mit $\theta = \arctan\left(\frac{G_y}{G_x}\right)$ berechnet. Es kann vorkommen, dass G_x den Wert Null besitzt. In diesem Fall wird der Winkel als 90 Grad angesetzt.
4. Nun wird auf die gesamte Intensitätsmatrix nach der Schritt 3 die Non-Maximum Suppression durchgeführt. Diese betrachtet nun in Richtung der maximalen Änderung (Schritt 3) die Intensitätswerte aus der Gradientenmatrix aus Schritt 2. Dabei wird entlang dieser Richtung das lokale Maximum erhalten und die restlichen Pixelwerte auf 0 gesetzt. Somit wird hier eine klare Kante errechnet (siehe Gleichung 3).

$$\text{NMS}(x, y) = \begin{cases} M(x, y) & \text{falls } M(x, y) \geq M(g_1) \text{ und } M(x, y) \geq M(g_2) \\ 0 & \text{sonst} \end{cases} \quad (3)$$

5. Nun werden zwei Schwellenwerte t_{high} und t_{low} festgelegt. Anhand dieser Schwellenwerte werden Pixel in drei Kategorien unterteilt:

- Keine Kante
- Schwache Kante
- Starke Kante

Canny klassifiziert schwache Kanten nur als Kante, wenn sie mit einem Pixel verbunden sind, der als starke Kante klassifiziert wurde oder über eine Kette schwacher Pixel eine Verbindung vorliegt. Pixel mit Wert unterhalb des Schwellenwerts werden ausgefiltert. So entsteht ein Bild, welches mit hellen Pixeln die Kanten des Rohbildes darstellt [2] [S.127ff].

7.3 Linienextraktion

Das gewonnene Bild wird nun zur Linienextraktion verwendet.

Allgemein werden zwei Verfahren für die Detektion von Liniensegmenten geprüft, welche oft in der digitalen Bildverarbeitung Verwendung finden. Diese sind zum einen die Hough Transformation und der Line Segment Detector (LSD) Algorithmus nach Rafael Grompone von Gioi et al.

7.3.1 Line Segment Detector

Der LSD Algorithmus ist ein geometrischer Extraktionsalgorithmus, welcher in linearer Zeit subpixel genaue Ergebnisse zu Liniensegmenten liefern kann. Auch der LSD basiert auf der Detektion von sich lokal stark ändernden Pixelwerten. Der LSD erstellt ein Vektorfeld, dessen Vektoren orthogonal zu den Richtungsvektoren der Gradientenänderung stehen (siehe cannyangle todo). Dies sieht dann wie folgt aus.

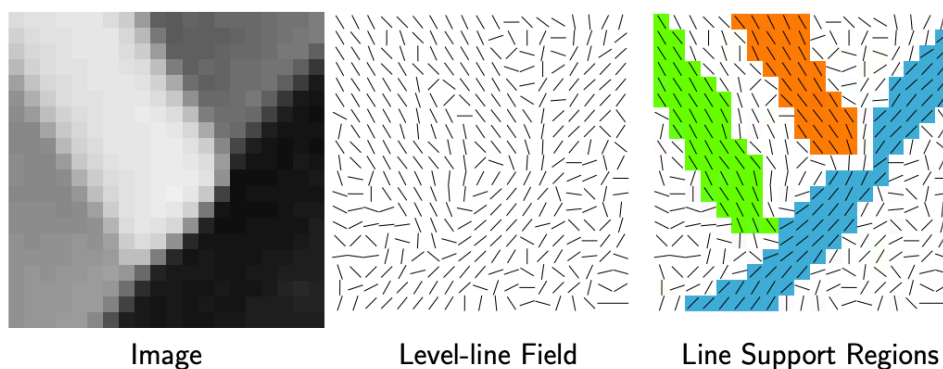


Abbildung 1 — LSD Vektorfeld

[3] Aus diesen Vektoren werden so genannte line support regions generiert die in Abbildung 1. Diese Regionen sind potentielle Linien. Um diese line support regions Wird ein Rechteck gelegt, dessen Ausrichtung entlang der Ausrichtung der Vektoren in diesem Feld liegt.

Nun wird jedes Rechteck durch das Helmholtz-Prinzip und der a contrario-Methode validiert. Nach dieser Methode wird ein Modell erstellt, welches in seiner Gesamtheit aus Rauschen besteht. Es wird nun angenommen, dass das Rechteck in solch einer Struktur nicht wiedergefunden werden kann. Das Rechteck wird validiert, wenn auf diesem Rausch-Modell nur eine sehr geringe Anzahl bis gar keine Ereignisse des selben Rechtecks auftreten [3] (S. 36f).

Ein großer Vorteil des LSD ist, dass er vollständig ohne das Anpassen von Parametern funktioniert und Parameter innerhalb des Algorithmus garnicht geändert werden können.

todo maybe mehr später falls gebraucht

$O(n)$

7.3.2 Hough Transformation

Auch die Hough-Transformation ist ein Algorithmus, welcher zur Detektion von Linien genutzt wird.

Innerhalb eines Bildes können alle geraden Linien als eine zwei-Parameter Familie gesehen werden. Folglich wird jede Linie mit zwei Parametern θ und ρ beschrieben. Es wäre naheliegend, eine Linie in der Form $y = m * x + b$ darzustellen. Diese Darstellung scheitert aber daran, dass vertikale Linien mit einer unendlichen Steigung beschrieben werden würden, was digitale Berechnungen kompliziert macht.

HOUGH RAUM Die einfache Darstellung einer Linie im Folgenden trotzdem genutzt, um den Hough Raum zu erklären. Man stelle sich eine Menge M_P an Punkten vor, die innerhalb eines Bilds auf einer Linie liegen. Diese Linie kann mit der oben beschriebenen Formel für eine Linie innerhalb des Bilds beschrieben werden. Dabei sind x_i und y_i beliebige Punkte auf der Linie, die aber vorerst unbekannt sind. Nun wird die Beschreibung der Linie zu $b = -m * x + y$ umgestellt. Nimmt man nun einen Punkt aus M_P und setzt ihn in die genannte Formel ein, erhält man eine Linie im Parameterraum (siehe Abbildung 2).

Diese Linie beschreibt nun alle Linien im Bildbereich, die diesen Punkt kreuzen. Dies wird nun für alle Punkte der Menge durchgeführt. Es ergibt sich ein Punkt (m, b) , welcher die gesuchte Linie im Bildraum darstellt, auf der alle Punkte der Menge M_P liegen. Nimmt man nun einen Punkt, der nicht auf der gesuchten Linie liegt, erhält man eine Linie im Parameterraum, die keinen gemeinsamen Schnittpunkt mit den anderen gefundenen Linien besitzt.

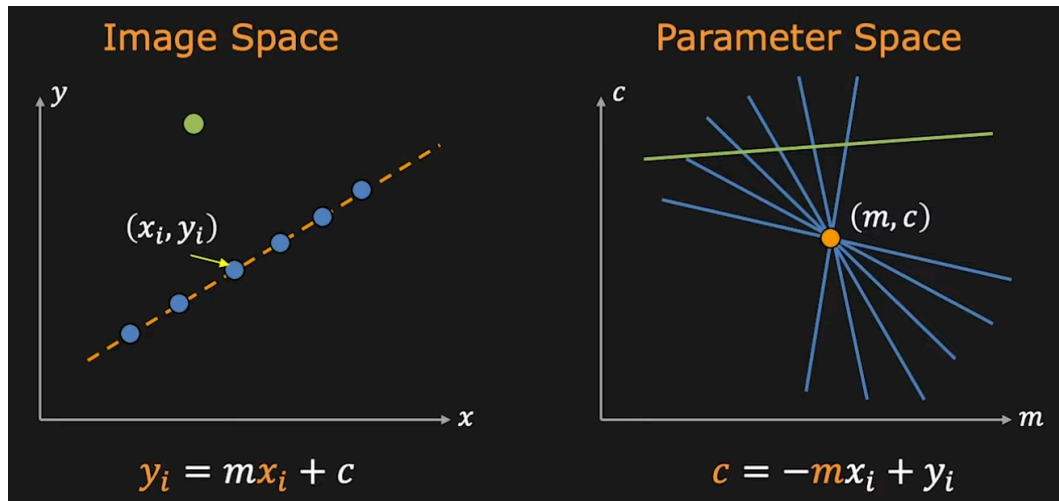


Abbildung 2 — Bild- und Parameterraum https://www.youtube.com/watch?v=XRBC_xkZREg

LINIENBESTIMMUNG Um nun innerhalb eines Bildes Linien zu erkennen, wird der jüngst beschriebene Hough-Raum verwendet. Dafür wird der Parameterraum P quantisiert. Dafür wird eine Matrix $A(m, b)$ über dem Parameterraum erstellt, die auch Akkumulator-Array genannt wird. Zu Beginn des Algorithmus wird dieses Array mit Nullen besetzt und sieht folglich so aus:

b					
	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0
	m				

Nun hat ein Kantenerkennungsalgorithmus Punkte innerhalb des Bildes erkannt hat, die potenzielle Linien sein könnten. Diese sind eine Menge K . Aus Effizienzgründen werden nicht die klassischen Geradenparameter m und b verwendet, sondern die Normalform mit den Parametern ρ (Abstand) und θ (Winkel).

Die Normalform einer Linie wird durch folgende Formel definiert:

$$\rho = x * \cos(\theta) + y * \sin(\theta)$$

Abbildung 3 – Normalform einer Geraden im Hough-Raum

Dabei ist:

- ρ der senkrechte Abstand von der Linie zum Ursprung (Betrag zwischen 0 und der Bilddiagonale)
- θ der Winkel des Normalenvektors der Linie zur x-Achse (typischerweise 0° bis 180°)

Der Vorteil dieser Parametrisierung ist, dass sie die gesamte Bildebene abdeckt und Singularitäten bei vertikalen Linien vermeidet, die bei der Steigungsform $m = \infty$ auftreten würde.

Ablauf des Hough-Transform-Algorithmus:

Für jeden erkannten Kantenpunkt (x_i, y_i) aus K werden alle möglichen Linien berechnet, die durch diesen Punkt verlaufen. Dies geschieht durch die Diskretisierung des θ -Bereichs in Schritte θ_j und das Berechnen der entsprechenden ρ -Werte nach obiger Formel. Im Akkumulator-Array (siehe oberes Diagramm) wird für jede berechnete Parameterkombination (ρ, θ) ein Zähler erhöht - ein Punkt „stimmt“ für die Parameter.

Nach der Verarbeitung aller Kantenpunkte enthält der Akkumulator Maxima an den Parameterkombinationen, die vielen Kantenpunkten entsprechen. Das untere Diagramm zeigt ein Beispiel mit 3 kollinearen Kantenpunkten: diese erzeugen Votes in vertikaler, horizontaler und diagonaler Richtung, die sich in der Mitte treffen und ein Maximum mit 3 Stimmen bilden. Diese Maxima repräsentieren die tatsächlich im Bild vorhandenen Linien.

Das gefüllte Akkumulator-Array nach der Verarbeitung könnte beispielsweise wie folgt aussehen:

$\uparrow b$					
	0	0	1	0	1
	0	0	1	1	0
	1	1	3	1	1
	0	1	1	0	0
	1	0	1	0	0
	$\rightarrow m$				

Die rot markierte Zelle mit dem Wert 3 zeigt das Maximum im Akkumulator-Array, das von drei kollinearen Kantenpunkten erzeugt wird. Diese drei Votes entstehen durch

die vertikale, horizontale und diagonale Ausrichtung der Kantenpunkte. Das detektierte Maximum (ρ_0, θ_0) wird als Parameterpaar für eine erkannte Linie verwendet. Nach der Verarbeitung aller Kantenpunkte können weitere Maxima ermittelt und damit mehrere Linien im Bild lokalisiert werden [4].

7.3.3 Pipeline für die Linienerkennung

Es wurden nun zwei Algorithmen zur Erkennung von Linien in einem Bild vorgestellt.

Durch praktische Tests hat sich der LSD Algorithmus dabei als die optimale Lösung für die Kreuzungserkennung erweisen, da dieser Algorithmus ohne weiteres Tuning von Parametern sehr zufriedenstellend funktioniert. Außerdem ist zu nennen, dass der LSD mit einer Zeitkomplexität von $O(n)$ deutlich effizienter funktioniert, als die Hough Transformation [5] (S.27). Somit wird für die Liniendetektion der Kreuzungserkennung der Canny Algorithmus mit nachgeschaltetem LSD Algorithmus gewählt.

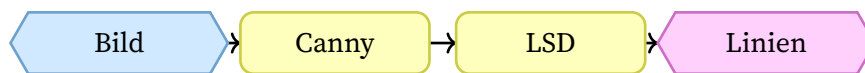


Abbildung 4 – Pipeline für Linienerkennung

7.4 Linienvorverarbeitung

7.4.1 Linienlängenfilter

Nachdem das Modul nun Linien aus einem Rohbild erkennen kann, sind noch einige Schritte nötig, um das Ergebnis der Linienerkennung besser nutzbar zu machen. Zum Einen ist es wichtig, zu kurze Linien aus der Menge der gefundenen Linien herauszufiltern. Das ist wichtig, da Haltelinien nach der Erkennung immer eine bestimmte Länge haben.

Eine Linie L sei innerhalb des Moduls als ein Vektor aus vier Werten dargestellt. Dieser Vektor L sei beschrieben durch $L = (x_1, y_1, x_2, y_2)$. Die Koordinaten innerhalb des Vektors stellen die Anfangs- und Endpunkte der Linie dar. Somit kann die Länge der Linie durch die euklidische Distanzformel $\text{length} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ berechnet werden. Es ergibt sich folgender Code.

python

```
1 def filter_by_length(self, lines, min_l: float = 70.0, max_l=10000):
2     res = []
3     for line in lines:
4         x1, y1, x2, y2 = line[0]
5         length = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
6         if length >= min_l and length <= max_l:
7             res.append(line)
8     return res
```

Euklidische Distanzformel zur Berechnung der Linienlänge

Abbildung 5 — Längenfiltration erkannter Linien

Es werden nur Linien mit einer Länge zwischen `min_l` und `max_l` behalten. Dies ist wichtig, um Rauschlinien auszuschließen und nur relevante Strukturen zu verarbeiten.

TODO: anfügen irgendwo, dass Variablenamen gekürzt werden aus Lesbarkeitsgründen

Durch Tests auf Sicht hat sich für einen ersten Längenfilter der Linien eine Pixellänge von 20 bewährt. Damit werden vorerst kleine Rauschlinien aus der potentiellen Menge entfernt.

Filterung der Linien ist wichtig, da eine geringe Menge an Linien dazu führt, dass das später zu implementierende System weniger Möglichkeiten hat, eine falsche Linie auszuwählen.

TODO: Bilder aus dem Modul aufnehmen und hier rein pasten

7.4.2 Region of Interest Filter

Der Region-of-Interest-Filter (ROI-Filter) beschränkt die verarbeiteten Linien auf einen definierten Bereich des Bildes. Dies ist besonders hilfreich, um irrelevante Bildbereiche auszuschließen und die Verarbeitung auf die Bereiche zu konzentrieren, in denen tatsächlich relevante Kreuzungen zu erwarten sind. Der ROI wird durch relative prozentuale Grenzen definiert, die automatisch auf die tatsächliche Bildgröße skaliert werden.

Das Konzept des ROI-Filters lässt sich algorithmisch wie folgt darstellen:

Algorithmus 1: ROI-basierte Linienfilterung

Eingabe: Liste von Linien, Bildhöhe und -breite, ROI-Grenzen (in %)

Ausgabe: Gefilterte Liste von Linien

1. Konvertiere relative ROI-Grenzen in absolute Pixelkoordinaten
2. Für jede Linie: Behalte sie nur, wenn beide Endpunkte in der ROI liegen
3. Gib die gefilterten Linien zurück

Für die relativen ROI-Grenzen wird durch Tests eine Definition von $\text{FILTERING_ROI_REL_RLTB} = (0.80, 0, 0, 0.815)$ gewählt.

TODO: auch hier evtl bild der roi

7.4.3 Linienfusion

Nachdem Linien nach Ort und Länge gefiltert wurden, ist beim Testen ein Problem zu beobachten. Durch Sichttests wird festgestellt, dass in vielen Fällen tatsächliche Haltelinien durch mehrere, unterbrochene Linien dargestellt werden, die logisch zu einer Linie im Bild gehören. Außerdem sind die zu erkennenden Linien stets gerade, es werden also keine Kurven erkannt. Deswegen ist es ein sinnvoller Schritt, die gefunden Linien zu fusionieren. Die Idee ist, Linien, die örtlich nebeneinander liegen und ähnliche Winkel haben, zu einer Linie zu fusionieren. Ziel ist es also, aus einer Menge an Linien eine einzelne repräsentative Linie zu finden.

Die Fusion der Linien läuft wie folgt ab:

Algorithmus 2: Linienfusion durch Greedy Clustering

Eingabe: Liste von Linien, Winkeltoleranz, Distanztoleranz

Ausgabe: Liste der fusionierten Linien

1. Berechne Winkel und Mittelpunkt jeder Linie
2. Gruppiere Linien mit ähnlichem Winkel und nah beieinander liegendem Mittelpunkt (Greedy Clustering)
3. Für jede Gruppe: Sammle alle Endpunkte, nutze PCA zur Hauptachsen-Berechnung
4. Projiziere Endpunkte auf die Hauptachse und finde die Extrempunkte

5. Diese Extrempunkte werden die Endpunkte der fusionierten Linie
6. Gib die Liste der fusionierten Linien zurück

Im Folgenden werden die Schritte des Algorithmus näher erläutert.

WINKEL- UND MITTELPUNKTSBERECHNUNG Um die Winkel und Mittelpunkte der Linien zu berechnen, wird eine Linie zuerst als Vektor dargestellt. Durch Differenzbildung zwischen den Start- und Endpunkten erhalten wir einen Linienvektor. Dieser sieht wie folgt aus: $l = \begin{pmatrix} x_2 - x_1 \\ y_2 - y_1 \end{pmatrix}$. Nun wird aus diesem Vektor mit `math.atan2(dy, dx)` der Winkel der Linie bestimmt. Der Mittelpunkt wird durch Division des Vektors mit Zwei bestimmt.

GRUPPIERUNG DER LINIEN Nun wird über alle Linien iteriert. Bei jeder Iteration i wird nun geprüft, ob diese Linie bereits überprüft wurde. Ist dies nicht der Fall, wird in einer weiteren Schleife im Bereich $[i + 1, n]$ der Differenzwinkel und die Differenzdistanz des Elements j zum Element i berechnet. Liegen diese Werte innerhalb eines vorgegebenen Toleranzbereichs, werden sie in eine Gruppe mit dem Element i hinzugefügt. Es bilden sich somit Gruppen aus Linien mit ähnlicher Ausrichtung, die nah beieinander liegen. Gruppen mit zu wenigen Mitgliedern werden gelöscht.

PRINCIPAL COMPONENT ANALYSIS Für jede Linie werden die Start- und Endpunkte bestimmt. Die zuvor bestimmten Gruppen werden somit also zu 2D-Punktemengen. Nun wird die Principal Component Analysis (PCA) verwendet, um die Gerade zu finden, die am besten durch die gesamte Punktwolke passt.

Um die PCA durchzuführen, müssen die Punkte vorverarbeitet werden. Zum Einen wird der Mittelpunkt aller Punkte bestimmt und anschließend jeder Punkt um diesen Mittelwert verschoben. Das hat zur Folge, dass der Mittelwert aller Punkte nun im Ursprung liegt [6] (S. 433). Nun wird die Single Value Decomposition (SVD) genutzt, um die Punktwolke in Hauptachsen zu zerlegen. Im Code wird dies mit folgendem Befehl durchgeführt: `U, S, Vt = np.linalg.svd(pts - mean)`. `Vt` liefert nun eine Matrix mit den Hauptachsen der Punktwolke. Darin ist der erste Vektor `Vt[0]` die Hauptachse der Datenpunkte, also die Achse „in der sich die Punkte am weitesten ausbreiten“. Die zweite Achse liegt dann orthogonal auf der Hauptachse [6] (S. 434ff). Es wird die erste Hauptachse genutzt, um die „Richtung“ der Punktwolke zu finden, die dann zum konstruieren der fusionierten Linie benutzt werden kann. Die Punkte aus der Punktwolke werden jetzt entlang der Hauptachse projiziert und anschließend die Extrema

P_1 und P_2 der Projektion gespeichert. Diese werden nun mit der Formel $p = \text{mean} + m * s$ also $P_{2d_max} = \text{mean} + m * P_1$ und $P_{2d_min} = \text{mean} + m * P_2$ zurück in den 2D-Raum gerechnet. Das sind dann die Punkte der fusionierten Linie. Diese fusionierte Linie wird nun einem Ergebnisarray angehängt.

7.4.4 Winkelfilter

Um einen weiteren Filter zu konzipieren, der die Menge an Linien, die zur Detektion von Haltelinien in Betracht fallen, minimiert, muss die Hauptausrichtung aller Haltelinien beachtet werden. Diese sind relativ zum Fahrzeug in erster Linie stets vertikal oder horizontal (TODO später auf Kurven verlinken). Somit ist es sinnvoll, einen Filter zu implementieren, der Linien anhand ihres Winkels ausschließen kann.

Der Winkelfilter iteriert über eine Menge an Linien und berechnet durch den Winkel des Differenzvektors des Start- und Endpunkts der Linie die Ausrichtung eben dieser. Dieser Winkel wird nun normalisiert und daraufhin getestet, ob dieser innerhalb einer Toleranz um 90 Grad bzw. 0 Grad liegt. Die Linien werden als zwei Listen horiz und vert zurückgegeben.

7.4.5 Pipeline für die Linienvorverarbeitung

Die Linienvorverarbeitung folgt einer festen Pipeline mit vier Filterschritten. Diese werden nacheinander auf die erkannten Linien angewendet, um die Menge an irrelevanten oder fehlerhaften Linien zu reduzieren:



Abbildung 6 – Pipeline für Linienvorverarbeitung und Filterung

Durch Testen dieser Pipeline haben sich außerdem einige Verbesserungsmöglichkeiten ergeben, die auch in die Pipeline integriert werden. So ist zu beobachten, dass durch Wiederholung der Fusionsschritte, zuerst mit einer höheren Winkeltoleranz (15 Grad) und Abstandstoleranz (100 Pixel) und danach mit einer geringen Winkeltoleranz (5 Grad) und höherer Abstandstoleranz (120 Pixel) repräsentativere Linien erzeugt werden können.

Es wurde auch das dreifache Fusionieren getestet, welches aber im Vergleich zu der zusätzlichen Laufzeit keine Verbesserungen, sondern eher Verschlechterungen zur Linienqualität gebracht hat.

7.5 Haltelinienklassifikation

Nach der Linienvorverarbeitung liegt nun eine Menge an Linien vor, die für die korrekte Detektion der Haltelinien in Frage kommen. Zuerst wird festgelegt, welche Haltelinien gesucht werden. Diese sind:

1. Haltelinie der eigenen Fahrspur (Egolinie)
2. Haltelinie der gegenüberliegenden Fahrspur (Opp-Linie für opposing lane)
3. Rechte Haltelinie
4. Linke Haltelinie

Es muss auch die Position und die Art der Linie bestimmt werden. Jede Linienart kann sowohl als durchgezogene Linie, als auch als gestrichelte Linie vorliegen.

7.5.1 Primitive Ermittlung der Ego- und Opplinie

In einem ersten Schritt liegt es nah, die Haltelinie der eigenen Spur zu detektieren. Diese erlaubt es dem Fahrzeug während der Caudri-Challenge rechtmäßig zum Halt zu kommen und gegebenenfalls andere Linien zu detektieren.

Als erster Ansatz wird die Methode gewählt, die Linie als Egolinie zu klasifizieren, die in ihrer Position am nächsten zum Fahrzeug ist und eine horizontale Ausrichtung hat. Um dies umzusetzen, wird der bereits implementierte Winkelfilter genutzt. Durch diesen erhalten wir eine Menge an Linien `horiz`, die alle horizontalen Linien in der Region of Interest darstellen. Nun wird über alle in dieser Menge enthaltenen Linien iteriert und diese dabei nach der Entfernung zur Mitte der untersten ROI Kante sortiert. Die Linie, die den geringsten Abstand aufweist, ist die Egolinie. Nach einer analogen Logik wird nun auch die Opplinie ermittelt, in dem nach der Linie mit geringsten Abstand zur oberen ROI kannte gesucht wird.

TODO Bild einfügen

Diese vorgestellte Methode liefert zufriedenstellende Ergebnisse in Kreuzungen, in denen nur eine Ego- und Opplinie vorhanden sind. Außerdem sollten die Lichtverhältnisse so ausgelegt sein, dass der LSD keine weiteren Linien erkennt, die um die Ego- oder Opplinie herumliegen. Ist dies der Fall, werden Linien falsch klassifiziert. Neben der genannten Schwäche nutzt diese Methodik nicht die allgemeine Struktur einer Kreuzung aus. Innerhalb dieser liegen alle Haltelinien in einem bestimmten, leicht variablen Abstand von einander innerhalb der Kreuzung. Die Kreuzung selbst kann als ein Quadrat interpretiert werden, welches entweder eine Kante (Halteline) oder keine Kante besitzen kann. Die Richtung, in der Spuren in die Kreuzung hineinfließen, ist mit Ausnahme der eigenen Spur, für die Kreuzungsklassifikation nicht relevant.

Mit Blick auf diese Erkenntnisse wird eine neue Methodik entwickelt, die es erlaubt, zuverlässiger die Ego- und Opplinie zu erkennen.

7.5.2 Kreuzungsmittenbasierte Detektion

TODO: Verweis zu Zentrumsberechnung einfügen Die Struktur einer Kreuzung legt nahe, dass Haltelinien stets relativ zueinander in einer ähnlichen Position liegen, die in Abschnitt 7.5.1 beschrieben ist. Auf Basis dieser Idee wird nun die Kreuzungsmitte genutzt, um Haltelinien zu detektieren.

In einem ersten Versuch, die Kreuzungsmitte zur Detektion der Haltelinien zu nutzen, wird anstatt der Kanten der ROI nun die Kreuzungsmitte als Anhaltspunkt für Abstandsberechnungen genutzt. Demnach sind die Linien Ego- und Opplinie, die am nächsten zur Kreuzungsmitte liegen.

TODO: irgendwann auch den Winkel der Linien erklären (prominent angle) und dazu auch die clip region

Diese Methodik weist gegenüber der primitiven Methodik einige wichtige Verbesserungen auf. Die wichtigste Verbesserung ist, dass die Erkennung nun nicht darauf gestützt ist, dass der Bildausschnitt genau so auf der Kreuzung liegt, dass Ego- und Opplinie die nächsten Linien zur ROI Unter- bzw. Oberkante sind. Das ist vor allem der Fall, wenn das Fahrzeug noch weiter von der Kreuzung entfernt ist oder der LSD eine Linie erkennt, die zwischen einer der ROI Kanten und der tatsächlichen Kreuzung liegt. Durch diese Methodik wird nun auch die Struktur einer Kreuzung ausgenutzt, in der Haltelinien relativ zueinander immer gleich angeordnet sind. Dies erzeugt eine robustere Haltelinien-detektion.

7.5.3 Sektorbasierte Detektion der rechten und linken Haltelinie

Nachdem nun vorerst die Ego- und die Opplinie erkannt werden, ist es wichtig, auch die Haltelinien der Spur, die von Egoausrichtung rechts in die Kreuzung einmündet und die Haltelinie der Spur links zu detektieren. Die Idee der Kreuzungsmitte legt nahe, dass diese auch zur Detektion der rechten und linken Haltelinie genutzt werden kann. Man berechne die vertikalen Linien, die am nächsten am Kreuzungszentrum sind. In der Entwicklung ist man nun aber mit der Schwierigkeit konfrontiert, dass die Kreuzungsmitte nicht zuverlässig perfekt zentral platziert ist. Dies hat zur Folge, dass oft Linien, die tatsächlich zur Spur gehören, als Haltelinien erkannt werden.

DREIECKSSEKTOR Aufgrund dieser Tatsache wird eine Methodik vorgestellt, die zwei dreieckige Sektoren nutzt, die innerhalb der ROI rechts und links platziert werden, um entsprechende Haltelinien zu finden. TODO: Bild einfügen. Diese Sektoren bestehen aus einem nach oben geöffneten Dreieck für die rechte Haltelinie und einem nach unten geöffneten Dreieck für die linke Haltelinie. Die Dreiecksstruktur ist darin begründet, dass bei minimalem Lenken des Fahrzeugs die Querhaltelinien in x-Position teilweise nach rechts oder links verschoben sein können.

Innerhalb des Sektors wird nun die Linie als Haltelinie ausgewählt, die am längsten ist.

Ein Problem dieser Sektorform ist, dass die Fläche, in der die Haltelinie gesucht wird, zur Grundseite des Dreiecks am größten ist. Das bewirkt, dass die Suchfläche (beispielsweise für die rechte Haltelinie) größer ist, je weiter die Kreuzung vom Fahrzeug entfernt ist. Das führt dazu, dass die tatsächliche Haltelinie nicht innerhalb der Suchregion liegt, wenn das Fahrzeug an der Kreuzung steht.

TRAPEZFÖRMIGER SEKTOR Um dem Umstand entgegenzuwirken, dass die Querhaltelinien nicht in der Suchregion liegen, wird der Sektor zu einer Trapezform geändert. Diese werden auf der rechten und linken Seite der ROI platziert. Diese Änderung hat zur Folge, dass der Suchbereich in der Mitte der Höhe der ROI am größten ist. Steht das Fahrzeug nun vor der Kreuzung, soll die Haltelinie theoretisch innerhalb des Sektors liegen.

TODO: Bild??

Nach weiteren Überlegungen und Tests wird auch dieser Ansatz verworfen, da statische Formen innerhalb der ROI, in denen die Haltelinien theoretisch zu erwarten sind, nicht zuverlässig funktionieren. Je nach Kalibrierung der Kamera und Anfahrt auf die Kreuzung, müssten sich die Sektoren in ihrer Position verändern.

QUADRANTENBASIERTE SUCHE Die ROI wird nun in vier Quadranten unterteilt, die von Q1 bis Q4 (1 - oben links, 2 - oben rechts, 3 - unten links, 4 - unten rechts) nummeriert sind. Innerhalb dieser Quadranten wird nun die rechte Haltelinie im Quadranten zwei oder im Quadranten 4 erwartet. Die linke Haltelinie wird im Quadranten eins erwartet. Der Quadrant drei wird für die linke Haltelinie nicht in Betracht gezogen, da durch die Kameraperspektive in diesen Quadranten keine nutzbaren Informationen (also kein nutzbares Bild) abgebildet sind.

Mit Hilfe der Quadrantenbasierten Suche gelingt es, Querhaltelinien zu erkennen.

TODO: wie und wann Sprung auf BEV erklären

7.5.4 Zentrumsbasierte Detektion aller Haltelinien

Die vorgestellten Methoden zur Detektion der rechten und linken Haltelinien weisen bei Kreuzungsanfahrt aus Kurven oder bei falsch detektierten Linien enorme Defizite auf.

Aufgrund dieses Umstands werden auch die Querhaltelinien auf Basis des Kreuzungszentrums berechnet. Es bleibt jedoch die Schwierigkeit, dass die aus dem Bilddaten berechnete Position des Kreuzungszentrums nicht exakt mit der tatsächlichen Position des Kreuzungszentrums übereinstimmt. Um, diesem Problem entgegenzuwirken, werden die aus dem Kreuzungszentrum abgeleiteten Referenzpunkte (TODO Kapitel referenzieren) genutzt. Mit Hilfe dieser Referenzpunkte ist es möglich, eine zuverlässige Detektion aller Haltelinien aus der Menge der vorverarbeiteten Linien durchzuführen.

Es wird auch die Hauptspurausrichtung genutzt, um die Referenzpunkte so zu berechnen, dass die Referenzen für die Ego- und Opplinie in Richtung des Spurwinkels liegen und die Referenzpunkte für die rechte und linke Haltelinie orthogonal dazu liegen (siehe dazu Abschnitt 7.6.5).

7.6 Berechnung des Kreuzungszentrums

Die Struktur der Kreuzungen innerhalb der Caudri-Challenge können zur verbesserten Detektion von Haltelinien ausgenutzt werden. Demnach liegen die Haltelinien jeder Kreuzung stets auf den Kanten eines Quadrats, dessen Ecken die Punkte sind, an denen die Spuren zur Kreuzung zusammenkommen.

7.6.1 Primitive Bestimmung der Kreuzungsmitte

Als erster Schritt liegt es nah, die Kreuzungsmitte als Zentrum der ROI festzulegen. Dies ist tatsächlich hilfreich, da es ein erster Schritt ist, die Struktur der Kreuzung auszunutzen und nicht Abstandsberechnung zu den Kanten der ROI zu nutzen. Allerdings weist diese Methodik das Problem auf, dass die Korrekten Haltelinien nur richtig erkannt werden, wenn das ROI-Zentrum gerade innerhalb der tatsächlichen Kreuzung liegt. Ist das Fahrzeug noch weiter von der Kreuzung entfernt oder ist schon zu Teilen in die Kreuzung hineingefahren, kann es dazu kommen, dass Linien außerhalb der Kreuzung als Haltelinien klassifiziert werden.

7.6.2 Algorithmische Kreuzungsmittendetektion

Bei weiterer Überlegung, die tatsächliche Kreuzungsmitte zu berechnen, wird ersichtlich, dass mehr Bildinformationen genutzt werden können, um das Zentrum verlässlicher zu berechnen.

Es ist denkbar, die Schnittpunkte der Spurlinien zu berechnen und daraufhin daraus das Kreuzungszentrum abzuleiten. Da bisher kein Ansatz ausgearbeitet wurde, um die Spuren aus dem Kamerabild zu berechnen, muss zuerst festgelegt werden, welche der Linien aus der gesamten Linienmenge potentielle Spuren sind.

DBSCAN Ein Ansatz, um Spurlinien zu erkennen und daraufhin die Kreuzungsmitte zu berechnen, ist der DBSCAN Algorithmus.

Der Algorithmus beruht auf der Idee, aus einem mehrdimensionalen Datensatz so genannte Cluster (dt. Haufen) an Datenpunkten zu finden, wobei die clusterinterne Ähnlichkeit maximiert und die Ähnlichkeit zwischen den Clustern minimiert werden soll [7] (S. 232). Ein Clustering Algorithmus wird dann als effektiv angesehen, wenn er viele der folgenden Kriterien abdeckt [7] (S. 232):

- Der Entwickler braucht minimale Domänenkenntnisse, um den Algorithmus zufriedenstellend zu parametrieren
- Der Algorithmus erkennt Cluster beliebiger Form
- Der Algorithmus arbeitet effizient, auch bei großen Datensätzen

Für den vorliegenden Anwendungsfall sind diese Kriterien auch wichtig, da der DBSCAN Algorithmus genutzt werden soll, um aus hellen Pixeln, die als Datenpunkte interpretiert werden, Cluster zu finden, die dann die Spuren darstellen.

Die Hauptidee des Algorithmus ist, dass jeder Punkt P einen anderen Punkt K innerhalb eines Radius ε um den Punkt aufweist. Durch dieses Kriterium ergibt sich ein so genannter Core Point. Dieser wird wie folgt beschrieben:

$$N_{\varepsilon}(P) > \text{MinPts}$$

Abbildung 7 — DBSCAN Core Point Kriterium

Die Parameter ε und MinPts können hierbei vom Anwender gewählt werden. ε stellt dabei den Radius dar, in dem eine Mindestanzahl MinPts an anderen Datenpunkten gefunden werden soll, damit P als Core Point klassifiziert wird. Der Algorithmus fährt dann fort und fügt jedem Core Point Datenpunkte aus der Umgebung zu. Wenn jedem Cluster kein neuer Punkt mehr hinzugefügt werden kann, terminiert der Algorithmus [7] (S. 233).

Innerhalb der Berechnung des Kreuzungszentrums soll der DBSCAN Algorithmus verwendet werden, um durch Clustering Linien zu bilden. Es stellt sich die Frage, ob anstatt des DBSCAN Algorithmus bereits verfügbare Linien genutzt werden können, die der Line Segment Detector berechnet. Der LSD weist dabei ein Problem auf, was durch den DBSCAN behoben werden kann, und zwar, dass Linien oft fragmentiert sind und dadurch keine eindeutige Kreuzung der Linien berechnet werden kann.

Nachdem Cluster berechnet wurden, kann durch PCA (TODO: Referenz hinzufügen), eine Menge an Linien gewonnen werden, die danach miteinander gekreuzt werden. Der Mittelpunkt aller Kreuzungen ist dann die Kreuzungsmitte.

Durch Testing in der Pipeline hat sich dieser Ansatz zwar als generell sinnvoll, aber nicht optimal herausgestellt. Da durch Fusionierung in der Vorverarbeitung bereits aus sehr kleinen Segmenten repräsentative Linien berechnet wurden, werden die weiteren Rechenkosten des DBSCAN als nicht gerechtfertigt angesehen. Zudem wies dieser Ansatz bei Anfahrt aus Kurven, in weiterer Entfernung von der Kreuzung und bei Mangel an erkannten Linien starke Mängel auf und berechnete die Kreuzungsmitte teilweise außerhalb der Kreuzung, was zu vollständig falschen Klassifikationen führte. Um den Parametrierungsaufwand zu minimieren, wurden vorerst weitere Ansätze getestet.

KREUZUNG VON LSD-LINIEN Um den Berechnungsaufwand durch DBSCAN zu umgehen, stellte sich der Ansatz, Linien aus der Vorverarbeitung zu nutzen, als sinnvoll dar. Demnach könne man eben diese Linien miteinander kreuzen und dann den Kreuzungsmittelpunkt berechnen.

TODO: ausführen bzw was schreibe ich hier rein Vielleicht sogar nochmal ausprobieren?? Sonst auslassen??

HARRIS CORNER DETECTION

Ein anderer Ansatz ist es, ohne Linien zu kreuzen innerhalb des Bildes Ecken zu detektieren und durch Mitteln aller Ecken das Kreuzungszentrum zu finden. Ein etablierter Algorithmus zur Detektion von Ecken ist der Harris Corner Detector.

Der Harris Corner Detector ist ein klassischer Eckenerkennung-Algorithmus, der 1988 von Chris Harris und Mike Stephens entwickelt wurde. Der Algorithmus identifiziert Punkte im Bild, an denen sich die Bildintensität in mehreren Richtungen stark ändert – typischerweise an Ecken oder anderen strukturierten Features.

Der Algorithmus berechnet zunächst die Gradienten des Bildes in horizontale und vertikale Richtungen. Für jeden Pixel wird dann eine Auto-Korrelations-Matrix (auch Struktur-Tensor genannt) berechnet, die misst, wie sich die Bildintensität in verschiedenen Richtungen ändert. Basierend auf dieser Matrix wird eine Eckenresponse-Funktion berechnet: $R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$, wobei λ_1 und λ_2 die Eigenwerte der Matrix sind und k ein Tuning-Parameter (typischerweise 0.04–0.06) ist. Pixel mit hohem Response-Wert werden als Eckenpunkte klassifiziert. Der Schwellwert wird meist empirisch gesetzt.

Ein großer Vorteil des Harris Detectors ist seine Recheneffizienz und die stabile Eckenerkennung in vielen Bildern. Allerdings hat der Algorithmus einige Schwächen: Die Eckenresponse ist nicht invariant gegenüber Skalierung, was bedeutet, dass unterschiedlich große Ecken unterschiedlich bewertet werden. Außerdem ist die Wahl des Parameters k kritisch und muss oft manuell angepasst werden.

SHI-TOMASI ECKENERKENNUNG

Neben dem Harris Corner Detector gibt es außerdem den Shi-Tomasi Algorithmus zur Erkennung von Ecken.

Der Shi-Tomasi Corner Detector ist eine Variante des Harris Corner Detectors, die 1994 von Jianbo Shi und Carlo Tomasi entwickelt wurde. Der Algorithmus versucht, Eckenpunkte in einem Bild zu erkennen, indem er lokale Bereiche analysiert, in denen sich die Bildintensität in mehreren Richtungen stark ändert.

Der Algorithmus funktioniert wie folgt: Zunächst wird für jeden Pixel die Auto-Korrelations-Matrix berechnet, die angibt, wie stark sich die Bildintensität in unterschiedliche Richtungen ändert. Diese Matrix wird als Struktur-Tensor oder Harris-Matrix bezeichnet. Anstatt der bei Harris verwendeten Eckenresponse-Funktion nutzt Shi-Tomasi direkt die Eigenwerte dieser Matrix: Ein Pixel wird als Eckenpunkt klassifiziert, wenn der kleinere der beiden Eigenwerte größer als ein Schwellwert ist. Dies bedeutet, dass die Bildintensität in mindestens zwei Richtungen stark variiert. Ein großer Vorteil dieses Ansatzes ist, dass er das optische Fluss-Tracking-Problem besser löst, da er auf die tatsächliche Struktur der Bildänderungen reagiert, anstatt auf eine heuristische Eckenmetrik.

Der Shi-Tomasi Algorithmus ist robuster gegen Skalierung und Rotation von Features, was ihn für die praktische Anwendung attraktiver macht. Außerdem berechnet der Algorithmus sowohl die Position als auch eine Qualitätsmetrik für jeden erkannten Eckenpunkt, was eine nachträgliche Filterung und Bewertung ermöglicht.

Durch Testing in der Pipeline wird der Shi-Tomasi Algorithmus als Basis der Kreuzungszentrumserkennung gewählt. Die Parametrierung des Algorithmus fällt simpel aus: Es werden für die Anzahl der auszugebenden Ecken $n = 4$ gewählt, sodass die repräsentativsten Ecken (die die Ecken der Kreuzung darstellen sollen) ausgegeben. Das Shi-Tomasi Zentrum ist dabei das Mittel der gefundenen Ecken.

7.6.3 Hybrider Ansatz zur Kreuzungsmittenberechnung

Obwohl Shi-Tomasi zufriedenstellende Ergebnisse liefert, kommt es auch zu Berechnungen von Kreuzungsmitten, die eine hohe Abweichung zum tatsächlichen Zentrum aufweisen. Um dem entgegenzuwirken, sollen Eckpunkte zuerst validiert werden, bevor ein Kreuzungszentrum berechnet wird.

Eine grundlegende Idee wäre es, zu prüfen, ob die Innenwinkelsumme der vier gefundenen Ecken 360° ist. Dieser Ansatz wird verworfen, da Eckpunkte in verschiedenen Anordnungen, die nicht unbedingt ein Viereck sein müssen, auch 360° ergeben. Stattdessen werden nun die einzelnen Winkel an den Ecken berechnet, die für ein Viereck jeweils 90° ergeben müssen. Um eine Metrik zu gewinnen, die die Ähnlichkeit der Anordnung der Ecken zu einem Viereck darstellt, werden die jeweiligen Fehler an jeder Ecke zu 90° berechnet und dann der Durchschnitt über alle vier Ecken gebildet. Anhand dieser Metrik kann nun bestimmt werden, ob das Shi-Tomasi Zentrum genutzt wird oder nicht.

Außerdem wird zusätzlich ein Gedächtnissystem eingebaut. Wird ein valides Shi-Tomasi Zentrum gefunden, wird dieses für fünf Frames gehalten. Es ändert dann in jedem Frame seine Position um einen festgelegten Pixelwert entlang der Hauptausrichtung der Linien (TODO: referenz zu dem hauptwinkel). Dieses Tracking erzeugt selbstverständlich keine Ergebnisse, die an die eines Kalman-Filters herankommen. Da die Anzahl der Frames, die das Zentrum gespeichert wird, aber relativ klein mit Blick auf die Framerate der Kamera ist, ist diese Näherung zufriedenstellend und liefert Ergebnisse, die gut für die Detektion von Linien genutzt werden können.

Kann kein Shi-Tomasi Zentrum berechnet werden, wird weiterhin das Zentrum der ROI als Redundanz angenommen.

7.6.4 Pipeline zur Kreuzungszentrumserkennung

Aus den gesammelten Erkenntnissen ergibt sich folgende Pipeline zur Erkennung des Kreuzungszentrums.

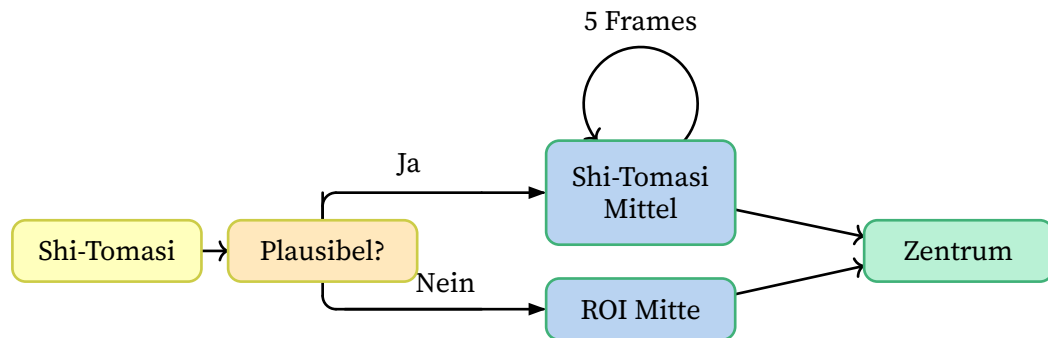


Abbildung 8 – Pipeline für Kreuzungszentrumserkennung

7.6.5 Erweiterung der Kreuzungsmitte

Bei der Kreuzungsmittenerkennung kommt es vereinzelt zu Berechnungen von Mitten, die nicht exakt in der tatsächlichen Mitte der Kreuzung liegen. Das ist allgemein unproblematisch für die Erkennung von Haltelinien, kann aber vereinzelt dazu führen, dass Haltelinien später erkannt werden als es möglich wäre oder das System in einigen Frames Haltelinien verliert. Grund dafür ist, dass aufgrund der Logik, die nächste Linie zum Kreuzungszentrum zu suchen, falsche Linien als Haltelinien klassifiziert werden. Die tatsächliche Haltelinie und eine sehr nah an dieser liegende „kämpfen“ um die Klassifikation als Haltelinie.

Um dieses Problem zu lösen, werden weitere Referenzpunkte aus der Kreuzungsmitte berechnet. Für die Ego- und Opplinie liegen diese entlang des Hauptwinkels der Linie und für die rechte und linke Haltelinie orthogonal dazu. Da die Referenzpunkte näher an dem tatsächlichen Bereich liegen, in denen die Haltelinien zu vermuten sind, werden auch die korrekten Haltelinien als diese klassifiziert. (TODO BILD)

7.7 Klassifizierung der Linienart

Neben der Lokalisation der Haltelinien muss auch erkannt werden, um welche Art es sich bei der haltelinie handelt. Haltelinien können sowohl gestrichelt als auch durchgezogen sein.

Um dies zu bestimmen, wird ein Algorithmus implementiert, der eine Linie und das Rohbild als Input erhält und den Weißanteil der Linie als auch die Anzahl der Lücken entlang der Linie ausgibt.

7.7.1 Linienausschnitt

Zu Beginn wird die Linienmitte bestimmt. Nun wird entlang des Linienzentrums rotiert, sodass die Linie waagerecht im Bild liegt. Dafür wird folgender Code genutzt.

python

```

1 h, w = image.shape[:2]
2 M = cv2.getRotationMatrix2D((mid_x, mid_y), angle, 1.0)
3 warped = cv2.warpAffine(image, M, (w, h), flags=cv2.INTER_LINEAR)

```

Die Linienmitten und der Linienwinkel wurden vorher berechnet

Abbildung 9 — Rotation des Bildes um das Linienzentrum

Die Funktion `cv2.getRotationMatrix2D` berechnet dabei eine Rotationsmatrix mit der außerdem eine Verschiebung möglich ist. Das ist nötig, da nicht um den Ursprung rotiert wird. Eine Rotationsmatrix um den Ursprung sieht wie folgt aus.

$$R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (4)$$

Da nun aber nicht um den Ursprung, sondern durch einen beliebigen Punkt im Bild rotiert wird die Matrix um konstante Werte erweitert, anhand derer alle Punkte um einen beliebigen Punkt im Bild verschoben werden können.

$$R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Nun wird das Zentrum der ursprünglichen Linie in das rotierte Bild transformiert.

Jetzt wird der Ausschnitt um die Linie aus dem rotierten Bild ausgeschnitten. Der Ausschnitt wird so ausgeschnitten, dass sowohl zu den Seiten als auch oben und unterhalb der Linie Abstand besteht. (TODO BILD einfügen)

7.7.2 Weißwertberechnung

Es ist unabdinglich, in verschiedenen Lichtverhältnissen zuverlässig den Weißwert der Linie berechnen zu können. Die Annahme ist, dass neben den weißen Pixeln des Ausschnitts auch graue oder nahezu weiße Pixel um den Linienabschnitt selbst liegen können. Dies kann etwa durch Sonneneinstrahlung passieren.

Um dem entgegenzuwirken, wird der Ausschnitt vorverarbeitet. Um das Bild unabhängig von Lichtverhältnissen nutzbar zu machen, müssen graue bis weiße Bildbereiche, die aber nicht zur Linie gehören, unterdrückt werden. Dafür wird zuerst der Canny-Algorithmus verwendet, um Pixel zu finden, die tatsächlich zur Linie gehören könnten. Daraus wird eine Pixelmaske generiert, die anschließend genutzt wird, um die originalen Pixelwerte der Pixel aus dem Bildabschnitt auszulesen. Es wird der Median dieser Pixelwerte gebildet. Falls der Canny-Algorithmus kein Ergebnis geliefert hat, wird der Median des ganzen Ausschnitts berechnet.

Der Ansatz, um für die Berechnung interessante Pixel, also die Pixel, die tatsächlich zur Linie gehören, gegenüber Hintergrundpixeln hervorzuheben, besteht in einer Verschiebung der Pixelwerte mit der Sigmoid-Funktion. Diese ist wie folgt beschrieben:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (6)$$

Abbildung 12 – Sigmoid Funktion

Mit der Sigmoidfunktion wird nun eine kontrastverstärkenden, nicht lineare Normalisierung der Pixelwerte innerhalb des Ausschnitts durchgeführt. Dafür werden zuerst die Pixelwerte normalisiert, indem sie zuerst um den Medianwert verschoben und dann durch 50 dividiert werden. Die Division hat den Hintergrund, dass die Sigmoid Funktion nur etwa im Bereich ± 3 effektiv normalisiert (siehe Abbildung 13). Das heißt, dass eine Intensitätsänderung um ± 150 zu einer vollständigen Sättigung in der Sigmoidfunktion führt, was für die Pixelwerte 0-255 gut passt.

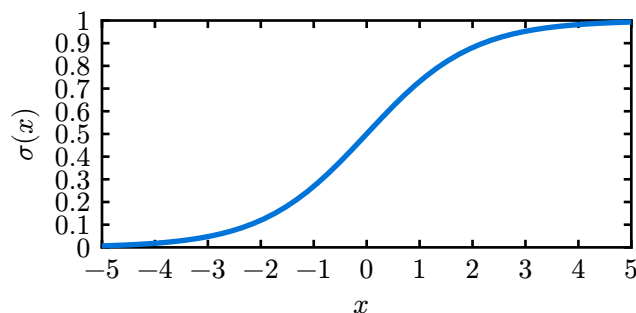


Abbildung 13 – Visualisierung der Sigmoid-Normalisierungsfunktion

Nun werden durch die Sigmoidfunktion Intensitätswerte, die nah am Median liegen in Richtung der 0.5 normalisiert. Werte, die weiter unter dem Median liegen, werden in Richtung 0 normalisiert. Werte über dem Median in Richtung 1 (Abbildung 13). Jetzt werden die berechneten Werte wieder mit Faktor 255 skaliert.

Im nächsten Schritt wird adaptives Thresholding genutzt, um die tatsächliche Segmentierung der Linie vom Hintergrund durchzuführen. Durch das adaptive Thresholding

wird noch einmal sichergestellt, dass Lichtinvarianzen nicht ins Gewicht fallen und das Liniensegment sicher erkannt wird, da es keinen globalen Threshold für ein Bild ermittelt. Das adaptive Thresholding nutzt lokale Thresholdermittlung für jeden Pixel.

Es werden nun gewonnene Informationen in einer einzigen Bitmaske zusammengefasst, die einen Minimalintensitätswert für die Linienpixel voraussetzt, die aber auch durch das adaptive Thresholding als zur Linie zugehörig klassifiziert wurden.

python

```
1 binary = (gray >= 60).astype(np.uint8) & adaptive_binary
```

Abbildung 14 – Bitmaske des Liniensegments für die Weißanteilberechnung

Nun wird das Bild in ein 1D Spaltenprofil umgewandelt, indem die Anzahl der weißen Pixel in jeder Spalte summiert werden (TODO: auch hier ein Bild einfügen). Mit Hilfe dieses Wertes und der Gesamtgröße des Ausschnitts wird nun der Weißanteil der Linie am Ausschnitt berechnet. Dieser Wert kann zur Validierung von Linien verwendet werden. Detektierte Linien, die tatsächlich aber nicht auf realen Linien liegen, weisen dabei einen sehr geringen Weißanteil im Ausschnitt auf und können somit ausgefiltert werden.

7.7.3 Gap-Detektion

Um zu bestimmen, ob es sich bei Linien um gestrichelte oder durchgezogene Halte-
linien handelt, werden die Lücken zwischen den weißen Teilen einer Linie berechnet. Im Schritt zuvor wurde ein Spaltenprofil des Ausschnitts angelegt, welches die Anzahl weißer Pixel pro Spalte aufsummiert anzeigt. Dieses Profil wird nun genutzt. Dabei werden die Spalten nach folgendem Algorithmus berechnet Abbildung 15.

```

1 Iteriere über die Spalten i des Spaltenprofils
2   Falls Wert bei i (Summe der weißen Pixel) == 0:
3     Falls bereits in einer Lücke, gap_size++
4     Falls noch in keiner Lücke
5       Setze in_gap auf 1
6   Falls Wert bei i > 0 Setze in_gap auf 0 und resette gap_size
7   Falls in_gap == 1 & gap_size > min_gap_size
8     gaps++
9 Falls nach Beenden der Schleife noch in_gap == 1, inkrementiere Gap-Anzahl
10 Gebe gaps zurück

```

Abbildung 15 – Lückenerkennungsalgorithmus

Schließlich wird über einen Parameter `min_gap_count` bestimmt, wann eine Linie gestrichelt ist. Durch Tests hat sich `min_gap_count=3` als optimaler Parameter für die Funktion ergeben.

7.7.4 Pipeline zur Klassifizierung der Linienart

Nachdem nun die algorithmische Basis der Klassifizierung der Linienart näher erläutert wurde, wird nochmal eine Übersicht über die genutzte Pipeline innerhalb des Moduls gegeben.

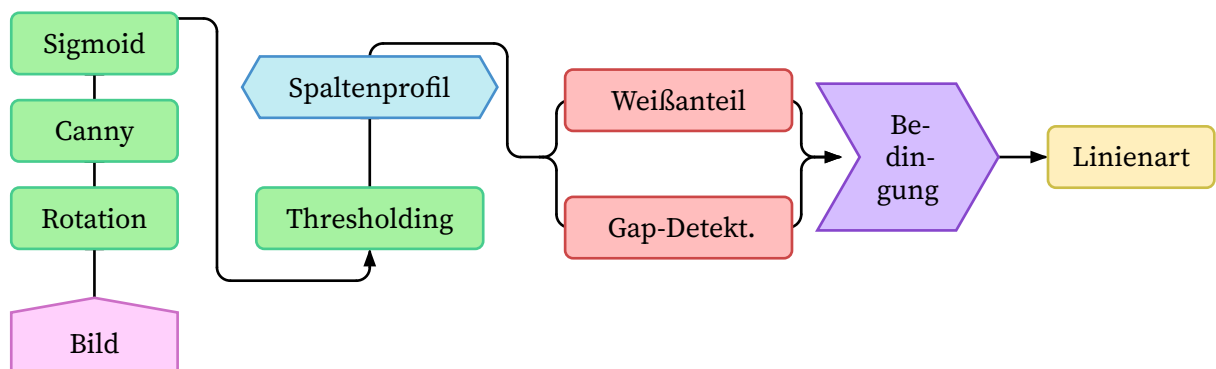


Abbildung 16 – Pipeline für die Klassifizierung der Linienart

7.8 Heading der Straße

7.9 Plausibilierung der Ego- und Opplinie

7.10 Plausibilierung der Querlinien

7.11 Ergebnisaggregator



Erläuterungen

Im folgenden werden einige nützliche Elemente und Funktionen zum Erstellen von Typst-Dokumenten mit diesem Template erläutert.

8.1 Ausdrücke und Abkürzungen

Verwende die `gls`-Funktion, um Ausdrücke aus dem Glossar einzufügen, die dann dorthin verlinkt werden. Ein Beispiel dafür ist:

Im diesem Kapitel wird eine [Softwareschnittstelle](#) beschrieben. Man spricht in diesem Zusammenhang auch von einem [Application Programming Interface \(API\)](#). Die Schnittstelle nutzt Technologien wie das [Hypertext Transfer Protocol \(HTTP\)](#).

Das Template nutzt das `glossarium`-Package für solche Glossar-Referenzen. In der zugehörigen [Dokumentation](#) werden noch weitere Varianten für derartige Querverweise gezeigt. Dort ist auch im Detail erläutert, wie das Glossar aufgebaut werden kann.

8.2 Listen

Es gibt Aufzählungslisten oder nummerierte Listen:

- Dies
- ist eine
- Aufzählungsliste

1. Und
2. hier wird

3. alles nummeriert.

8.3 Abbildungen und Tabellen

Abbildungen und Tabellen (mit entsprechenden Beschriftungen) werden wie folgt erstellt.

8.3.1 Abbildungen



Abbildung 17 — Eine Abbildung

8.3.2 Tabellen

	Area	Parameters
cylinder.svg	$\pi h \frac{D^2 - d^2}{4} \quad (7)$	h : height D : outer radius d : inner radius
tetrahedron.svg	$\frac{\sqrt{2}}{12} a^3 \quad (8)$	a : edge length

Tabelle 1 — Eine Tabelle

8.4 Programm Quellcode

Quellcode mit entsprechender Formatierung wird wie folgt eingefügt:

```
1  const ReactComponent = () => {  
2    return (  
3      <div>  
4        <h1>Hello World</h1>  
5      </div>  
6    );  
7  };  
8  
9  export default ReactComponent;
```

Listing 1 — Ein Stück Quellcode

8.5 Verweise

Für Literaturverweise verwendet man die `cite`-Funktion oder die Kurzschreibweise mit dem `@`-Zeichen:

- `#cite(form: "prose", <iso18004>)` ergibt:
International Organization for Standardization [8]
- Mit `@iso18004` erhält man: [8]

Tabellen, Abbildungen und andere Elemente können mit einem Label in spitzen Klammern gekennzeichnet werden (die Tabelle oben hat z.B. das Label `<table>`). Sie kann dann mit `@table` referenziert werden. Das ergibt im konkreten Fall: Tabelle 1



Fazit

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defenza et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defenza et.

A Literatur

- [1] Z. Xu, X. Baojie, und W. Guoxin, „Canny edge detection based on Open CV“, in *2017 13th IEEE international conference on electronic measurement & instruments (ICEMI)*, 2017, S. 53–56.
- [2] H. Agrawal und K. Desai, „Canny edge detection: A comprehensive review“, *International Journal of Technical Research & Science*, Bd. 9, Nr. Spl, S. 27–35, 2024.
- [3] R. Grompone von Gioi, J. Jakubowicz, J.-M. Morel, und G. Randall, „LSD: a Line Segment Detector“, *Image Processing On Line*, Bd. 2, S. 35–55, 2012.
- [4] R. O. Duda und P. E. Hart, „Use of the Hough transformation to detect lines and curves in pictures“, *Commun. ACM*, Bd. 15, Nr. 1, S. 11–15, Jan. 1972, doi: [10.1145/361237.361242](https://doi.org/10.1145/361237.361242).
- [5] D. Meng, W. Long, L. Hu, und L. Jiang, „Review on line detection of wood panel images“, *Advances in Engineering Innovation*, Bd. 5, S. 24–32, 2024.
- [6] H. Abdi und L. J. Williams, „Principal component analysis“, *WIREs Computational Statistics*, Bd. 2, Nr. 4, S. 433–459, 2010, doi: <https://doi.org/10.1002/wics.101>.
- [7] K. Khan, S. U. Rehman, K. Aziz, S. Fong, und S. Sarasvady, „DBSCAN: Past, present and future“, in *The Fifth International Conference on the Applications of Digital Information and Web Technologies (ICADIWT 2014)*, 2014, S. 232–238. doi: [10.1109/ICADIWT.2014.6814687](https://doi.org/10.1109/ICADIWT.2014.6814687).
- [8] International Organization for Standardization, „ISO/IEC 18004: Information technology – Automatic identification and data capture techniques – QR code bar code symbology specification“, in *ISO/IEC 18004:2015*, 2015.

B Glossar

API – Application Programming Interface [33](#)

HTTP – Hypertext Transfer Protocol [33](#)

Softwareschnittstelle: Ein logischer Berührungspunkt in einem Softwaresystem: Sie ermöglicht und regelt den Austausch von Kommandos und Daten zwischen verschiedenen Prozessen und Komponenten. [33](#)

Sperrvermerk

Die vorliegende Arbeit mit dem Titel

Entwicklung eines klassischen Kreuzungsdetektors

enthält unternehmensinterne bzw. vertrauliche Informationen der Robert Bosch GmbH, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Informatik der Duale Hochschule Baden-Württemberg Stuttgart vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (Robert Bosch GmbH) vorliegt.

Karlsruhe 22.03.2026

Reinhold Brant

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Entwicklung eines klassischen Kreuzungsdetektors

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Karlsruhe 22.03.2026

Reinhold Brant